Exercise 3 — EDF and RM Scheduling with Three Inputs: A Complete Hand Solution

Step-by-step printable hand solution of EDF and RM scheduling for a three-task periodic set

Exercise EDF and RM scheduling with three inputs and a hand solution
Course Advanced Operating Systems
Topic Real-time scheduling of periodic tasks
Task set T1(C=1,T=4,D=4), T2(C=3,T=8,D=8), T3(C=4,T=12,D=12)
Total utilisation U = 23/24 ~ 0.9583
Deliverable Persian and English reports (DOCX and PDF) + runnable code + blank worksheets
Repository github.com/kaspian021/Multilevel-Queue-Scheduling
Branch Exercise-3-EDF-RM-Manual-Scheduling

Table of contents

3 Abstract, keywords and report map
4 Introduction, problem statement and goals
5 Task-set design and preliminary arithmetic
6 RM priority computation and the EDF selection rule
7 Analytic tests before scheduling
8 The EDF analytic test and the prediction before running
9 Release pattern and deadlines
10 Hand solution for EDF (part 1): ticks 0 to 11
11 Hand solution for EDF (part 2): ticks 12 to 23
12 Gantt chart and EDF analysis
13 Hand solution for RM (part 1): ticks 0 to 11
14 Hand solution for RM (part 2): ticks 12 to 23
15 Gantt chart and the deadline miss under RM
16 Comparing the two schedules
17 Utilisation, idle time and processor occupancy
18 Reconciling the hand solution with the code
19 Scope of the results
20 Conclusion
21 References and reproduction commands
22 Blank worksheet 1: rewriting the EDF schedule by hand
23 Blank worksheet 2: EDF continued and the RM schedule
24 Blank worksheet 3: RM continued and the deadline checklist

Abstract, keywords and report map

Abstract

This exercise designs a three-task periodic set and solves its scheduling under EDF and RM by hand, step by step. Every calculation was first worked out on paper and then cross-checked against a Python program; the program output and the hand solution agree exactly.

The set is T1(C=1,T=4,D=4), T2(C=3,T=8,D=8) and T3(C=4,T=12,D=12). Total utilisation is 0.9583, i.e. 23/24, and the hyperperiod is 24. The analytic test shows that EDF runs this set without a single deadline miss, while response-time analysis for RM gives R(T3)=14, which exceeds the deadline of 12. Simulation reproduces exactly that one miss at t=12.

Keywords

Real-time scheduling; EDF; RM; absolute deadline; period; hyperperiod; response-time analysis; utilisation; deadline miss; idle time; hand solution.

Report map

- Sections 1-4: task-set design, preliminary arithmetic and priorities.
- Sections 5-8: analytic tests and selection rules.
- Sections 9-15: step-by-step EDF solution and its analysis.
- Sections 16-21: step-by-step RM solution and the deadline miss.
- Sections 22-28: comparison, hand arithmetic and the limits of the claim.
- Sections 29-30: blank worksheets for rewriting the solution by hand.

Introduction, problem statement and goals

Problem statement

Three periodic tasks run on a single preemptive processor. Each task is released at multiples of its period and every job must finish by its relative deadline. The goal is to determine the execution order under EDF and RM, compute the metrics and check for deadline misses.

Goals

- Design a task set with parameters that can be computed by hand.
- Compute the fixed RM priorities from the periods.
- Apply the earliest-deadline-first selection rule.
- Produce a step table for every tick of the hyperperiod.
- Draw the Gantt chart for both algorithms.
- Compute utilisation, idle time and deadline misses.
- Reconcile the hand solution with the Python program.

Model rules

- Single processor, preemptive scheduling.
- Time is discrete; one tick consumes one unit of execution.
- All tasks are released in phase at t=0.
- EDF tie rule: on equal absolute deadlines the smaller task index wins.
- A job that misses its deadline is not aborted; it runs to completion.
- The analysis horizon is one complete hyperperiod.

Scope: This solution shows the behaviour of the set over one hyperperiod. The definitive schedulability verdict for all time comes from the analytic tests presented in the following pages.

Task-set design and preliminary arithmetic

Task parameters

Task	C (execution)	T (period)	D (relative deadline)	Release instants
T1	1	4	4	0, 4, 8, 12, 16, 20
T2	3	8	8	0, 8, 16
T3	4	12	12	0, 12

Table 1. Parameters of the three periodic tasks

Utilisation

$$U = C1/T1 + C2/T2 + C3/T3$$
$$= 1/4 + 3/8 + 4/12$$
$$= 6/24 + 9/24 + 8/24 = 23/24 = 0.9583$$

Hand computation of total utilisation

Hyperperiod

$$H = \mathrm{lcm}(4, 8, 12) = 24$$

Hand computation of the hyperperiod

Total load in one hyperperiod

- T1 is released 6 times in 24 units and carries 6 x 1 = 6 units of load.
- T2 is released 3 times and carries 3 x 3 = 9 units.
- T3 is released 2 times and carries 2 x 4 = 8 units.
- Total load is 23 units, so in 24 time units there is exactly 1 unit of idle time.

Quantity	Value
Total utilisation U	0.9583 (23/24)
Hyperperiod H	24
Total load in the hyperperiod	23 units
Idle time	1 unit
Jobs in the hyperperiod	11 jobs

Table 2. Summary of the preliminary arithmetic

RM priority computation and the EDF selection rule

Fixed priority in RM

In RM the priority is derived from the period: a shorter period means a higher priority. The priority is computed at design time and never changes at run time.

Task	Period	RM priority	Rank
T1	4	highest	1
T2	8	medium	2
T3	12	lowest	3

Table 3. Fixed RM priorities for this task set

The selection rule in EDF

Under EDF, at every instant the job with the earliest absolute deadline is selected. The absolute deadline of job k of task i is k*T + D.

```
d(k, i) = k * T_i + D_i
choose the ready job with the smallest d(k, i)
tie rule: smaller task index first
```

Absolute deadline and the EDF selection rule

Absolute deadlines over one hyperperiod

Task	job 0	job 1	job 2	job 3	job 4	job 5
T1	4	8	12	16	20	24
T2	8	16	24	-	-	-
T3	12	24	-	-	-	-

Table 4. Absolute deadlines of all jobs in [0,24)

- At t=0 three jobs with deadlines 4, 8 and 12 are ready; EDF picks the nearest, T1.
- At t=12 the deadline of T3_0 arrives, while T1_3 with deadline 16 has just been released.
- EDF continues T3_0 at that instant, but RM preempts it because T1 has higher priority.

Analytic tests before scheduling

The Liu and Layland sufficient bound for RM

```
                    U <= n * (2^(1/n) - 1)
        n = 3  ->  bound = 3 * (2^(1/3) - 1) = 0.7798
    U = 0.9583 > 0.7798   =>   the bound gives no answer
```

Checking the Liu and Layland sufficient bound

- Utilisation exceeds the bound, but that does not mean failure.
- The condition is only sufficient; many sets above it are still schedulable.
- A definite answer needs response-time analysis.

RM response-time analysis

```
R_i = C_i + sum over j in hp(i):  ceil(R_i / T_j) * C_j
```

The response-time recurrence

Task	C	iter 1	iter 2	iter 3	final R	D	verdict
T1	1	1	-	-	1	4	ok
T2	3	4	4	-	4	8	ok
T3	4	8	13	14	14	12	MISS

Table 5. Hand execution of response-time analysis for the three tasks

- R(T1) = 1 <= 4, so T1 always finishes in time.
- R(T2) = 3 + ceil(4/4)*1 = 4 <= 8, so T2 is safe.
- R(T3) = 4 + ceil(14/4)*1 + ceil(14/8)*3 = 4 + 4 + 6 = 14 > 12, so T3 misses.
- Analytic verdict: this set is NOT schedulable under RM.

The EDF analytic test and the prediction before running

The utilisation condition for EDF

Because D equals T for every task in this set (implicit deadlines), the condition U <= 1 is both necessary and sufficient for EDF. No more elaborate test is required.

```
U = 0.9583 <= 1   =>   EDF can schedule this task set
```

Necessary and sufficient EDF condition for implicit deadlines

Prediction before running

Test	Result	Prediction
Liu and Layland bound (RM)	inconclusive	a sharper test is needed
Response-time analysis (RM)	R(T3)=14 > 12	exactly one deadline miss
Utilisation condition (EDF)	satisfied	no deadline miss

Table 6. Summary of the analytic predictions before simulation

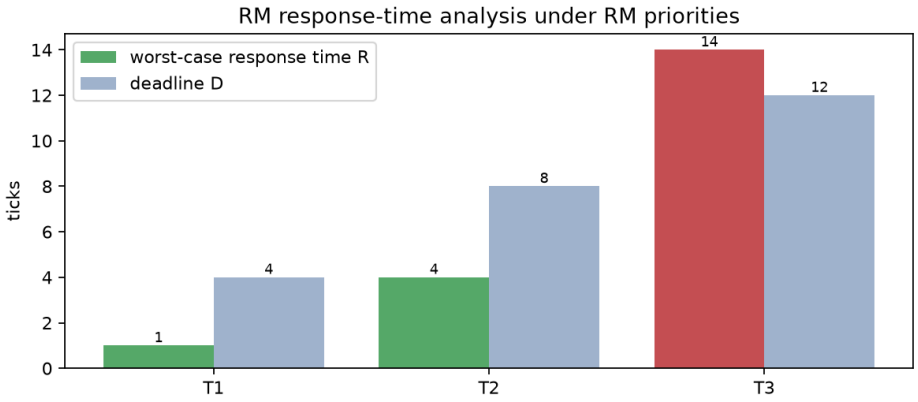


Figure 7. Response-time analysis chart: computed R against the deadline D

- In this chart the T3 bar crosses the deadline line.
- The figure announces the result we are about to see in the RM Gantt chart.
- Agreement between analytic prediction and simulation is the strongest evidence of correctness.

Release pattern and deadlines

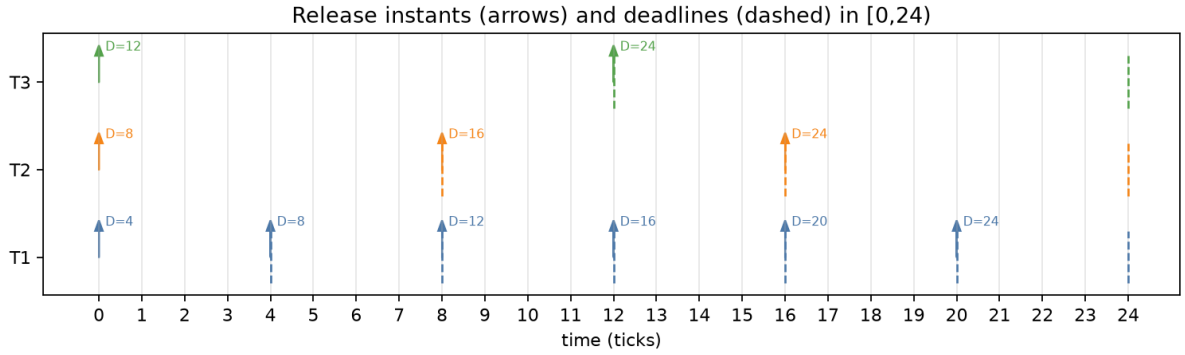


Figure 8. Release instants (arrows) and deadlines (dashed lines) in [0,24)

Reading the figure

- T1 is released at 0, 4, 8, 12, 16 and 20 and has 4 units of deadline each time.
- T2 is released at 0, 8 and 16 and has 8 units of deadline each time.
- T3 is released at 0 and 12 and has 12 units of deadline each time.
- At t=0 all three tasks are released together: the worst-case release pattern.
- At t=24 all tasks are in phase again, so one hyperperiod of analysis suffices.

Why being in phase matters

- Simultaneous release puts the heaviest load on the scheduler.
- In real-time analysis this is normally taken as the worst case.
- If a set is schedulable in this case, it is schedulable in the others.

Hand solution for EDF (part 1): ticks 0 to 11

In every row the ready set is shown with the absolute deadline and the remaining execution time; the job name T1_0 means job zero of task T1.

Interval	Ran	Ready jobs (d=deadline, rem=remaining)	Reason
0-1	T1_0	T1_0(d=4,rem=1), T2_0(d=8,rem=3), T3_0(d=12,rem=4)	d=4
1-2	T2_0	T2_0(d=8,rem=3), T3_0(d=12,rem=4)	d=8
2-3	T2_0	T2_0(d=8,rem=2), T3_0(d=12,rem=4)	d=8
3-4	T2_0	T2_0(d=8,rem=1), T3_0(d=12,rem=4)	d=8
4-5	T1_1	T1_1(d=8,rem=1), T3_0(d=12,rem=4)	d=8
5-6	T3_0	T3_0(d=12,rem=4)	d=12
6-7	T3_0	T3_0(d=12,rem=3)	d=12
7-8	T3_0	T3_0(d=12,rem=2)	d=12
8-9	T1_2	T1_2(d=12,rem=1), T2_1(d=16,rem=3), T3_0(d=12,rem=1)	d=12; tie, lower task index
9-10	T3_0	T2_1(d=16,rem=3), T3_0(d=12,rem=1)	d=12
10-11	T2_1	T2_1(d=16,rem=3)	d=16
11-12	T2_1	T2_1(d=16,rem=2)	d=16

Table 9. Step-by-step EDF table from t=0 to t=11

Hand solution for EDF (part 2): ticks 12 to 23

Interval	Ran	Ready jobs (d=deadline, rem=remaining)	Reason
12-13	T1_3	T1_3(d=16,rem=1), T2_1(d=16,rem=1), T3_1(d=24,rem=4)	d=16; tie, lower task index
13-14	T2_1	T2_1(d=16,rem=1), T3_1(d=24,rem=4)	d=16
14-15	T3_1	T3_1(d=24,rem=4)	d=24
15-16	T3_1	T3_1(d=24,rem=3)	d=24
16-17	T1_4	T1_4(d=20,rem=1), T2_2(d=24,rem=3), T3_1(d=24,rem=2)	d=20
17-18	T2_2	T2_2(d=24,rem=3), T3_1(d=24,rem=2)	d=24; tie, lower task index
18-19	T2_2	T2_2(d=24,rem=2), T3_1(d=24,rem=2)	d=24; tie, lower task index
19-20	T2_2	T2_2(d=24,rem=1), T3_1(d=24,rem=2)	d=24; tie, lower task index
20-21	T1_5	T1_5(d=24,rem=1), T3_1(d=24,rem=2)	d=24; tie, lower task index
21-22	T3_1	T3_1(d=24,rem=2)	d=24
22-23	T3_1	T3_1(d=24,rem=1)	d=24
23-24	IDLE		no ready job

Table 10. Step-by-step EDF table from t=12 to t=23

Gantt chart and EDF analysis

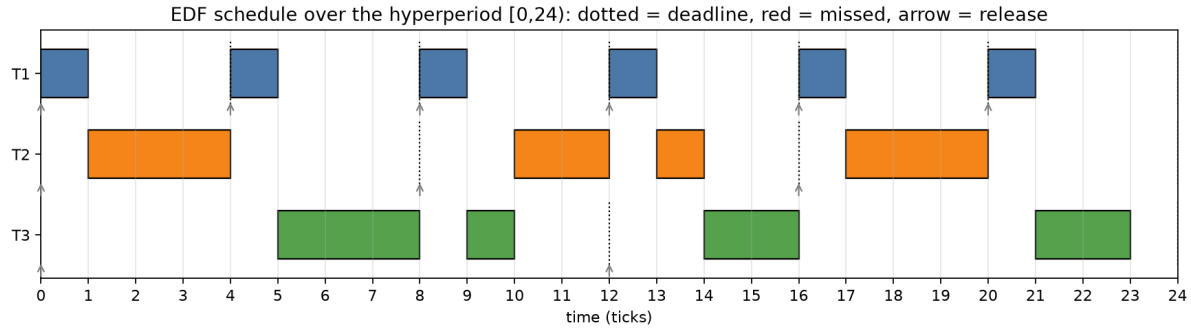


Figure 11. EDF Gantt chart over one hyperperiod; dashed lines are deadlines, arrows releases

Resulting schedule

```
t :  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
   : T1 T2 T2 T2 T1 T3 T3 T3 T1 T3 T2 T2 T1 T2 T3 T3 T1 T2 T2 T2 T1 T3 T3  -
```

EDF execution order over 24 ticks (the last tick is idle)

Checking the deadlines

Job	Release	Deadline	Finish	Response	Verdict
T1_0	0	4	1	1	on time
T2_0	0	8	4	4	on time
T3_0	0	12	11	11	on time
T1_1	4	8	5	1	on time
T1_2	8	12	9	1	on time
T2_1	8	16	13	5	on time
T1_3	12	16	13	1	on time
T3_1	12	24	19	7	on time

Table 12. Hand check of EDF deadlines (first eight jobs)

- No job missed its deadline.
- The only idle time is at tick 23, because the total load is 23 units.
- Utilisation is 23/24 = 95.83%.

Hand solution for RM (part 1): ticks 0 to 11

Under RM the priority is fixed, so the reason column always refers to the period of the task, never to its absolute deadline.

Interval	Ran	Ready jobs (d=deadline, rem=remaining)	Reason
0-1	T1_0	T1_0(d=4,rem=1), T2_0(d=8,rem=3), T3_0(d=12,rem=4)	T=4; fixed priority
1-2	T2_0	T2_0(d=8,rem=3), T3_0(d=12,rem=4)	T=8; fixed priority
2-3	T2_0	T2_0(d=8,rem=2), T3_0(d=12,rem=4)	T=8; fixed priority
3-4	T2_0	T2_0(d=8,rem=1), T3_0(d=12,rem=4)	T=8; fixed priority
4-5	T1_1	T1_1(d=8,rem=1), T3_0(d=12,rem=4)	T=4; fixed priority
5-6	T3_0	T3_0(d=12,rem=4)	T=12; fixed priority
6-7	T3_0	T3_0(d=12,rem=3)	T=12; fixed priority
7-8	T3_0	T3_0(d=12,rem=2)	T=12; fixed priority
8-9	T1_2	T1_2(d=12,rem=1), T2_1(d=16,rem=3), T3_0(d=12,rem=1)	T=4; fixed priority
9-10	T2_1	T2_1(d=16,rem=3), T3_0(d=12,rem=1)	T=8; fixed priority
10-11	T2_1	T2_1(d=16,rem=2), T3_0(d=12,rem=1)	T=8; fixed priority
11-12	T2_1	T2_1(d=16,rem=1), T3_0(d=12,rem=1)	T=8; fixed priority

Table 13. Step-by-step RM table from t=0 to t=11

Hand solution for RM (part 2): ticks 12 to 23

Interval	Ran	Ready jobs (d=deadline, rem=remaining)	Reason
12-13	T1_3	T1_3(d=16,rem=1), T3_0(d=12,rem=1), T3_1(d=24,rem=4)	T=4; fixed priority
13-14	T3_0	T3_0(d=12,rem=1), T3_1(d=24,rem=4)	T=12; fixed priority
14-15	T3_1	T3_1(d=24,rem=4)	T=12; fixed priority
15-16	T3_1	T3_1(d=24,rem=3)	T=12; fixed priority
16-17	T1_4	T1_4(d=20,rem=1), T2_2(d=24,rem=3), T3_1(d=24,rem=2)	T=4; fixed priority
17-18	T2_2	T2_2(d=24,rem=3), T3_1(d=24,rem=2)	T=8; fixed priority
18-19	T2_2	T2_2(d=24,rem=2), T3_1(d=24,rem=2)	T=8; fixed priority
19-20	T2_2	T2_2(d=24,rem=1), T3_1(d=24,rem=2)	T=8; fixed priority
20-21	T1_5	T1_5(d=24,rem=1), T3_1(d=24,rem=2)	T=4; fixed priority
21-22	T3_1	T3_1(d=24,rem=2)	T=12; fixed priority
22-23	T3_1	T3_1(d=24,rem=1)	T=12; fixed priority
23-24	IDLE		no ready job

Table 14. Step-by-step RM table from t=12 to t=23

Gantt chart and the deadline miss under RM

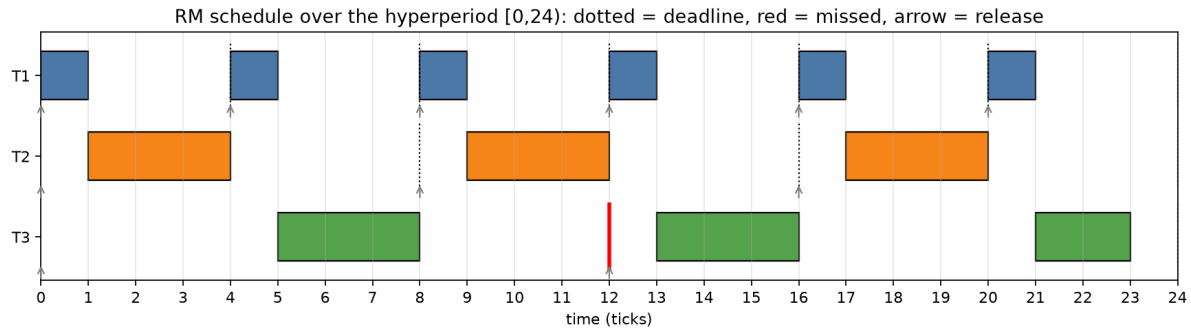


Figure 15. RM Gantt chart; the red line marks the missed deadline of T3_0

Resulting schedule

```
t :  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
   : T1 T2 T2 T2 T1 T3 T3 T3 T1 T2 T2 T2 T1 T3 T3 T3 T1 T2 T2 T2 T1 T3 T3  -
```

RM execution order over 24 ticks

The critical instant: t=12

- T3_0 was released at t=0 and ran during ticks 5, 6 and 7; one unit remained.
- At t=8 task T2_1 is released with higher priority and runs for three ticks.
- At t=12 task T1_3 is released and, having a shorter period, runs immediately.
- As a result the last unit of T3_0 runs at tick 13 and its finish time is 14.
- The deadline of T3_0 is 12, so it is 2 units late.

Job	Release	Deadline	Finish	Tardiness	Verdict
T3_0	0	12	14	2	MISS
T3_1	12	24	20	0	on time

Table 16. The T3 jobs under RM and their deadline status

Comparing the two schedules

Metric	EDF	RM
Deadline misses	0	1
Job that missed	-	T3_0 (deadline 12, finish 14)
Idle time	1	1
Utilisation	95.83%	95.83%
Context switches	12	11
Average response of T1	1.000	1.000
Average response of T2	4.667	4.000
Average response of T3	10.500	12.500
Maximum response	11	14

Table 17. Measured results of the two algorithms over one hyperperiod

Comparative analysis

- RM gives a better average response for T2: 4.000 against 4.667.
- EDF gives a better average response for T3: 10.500 against 12.500.
- But the primary criterion in real-time scheduling is deadline misses, not averages.
- So despite RM's better average for T2, the overall verdict favours EDF.
- Context-switch counts are close, so overhead is not the deciding factor.

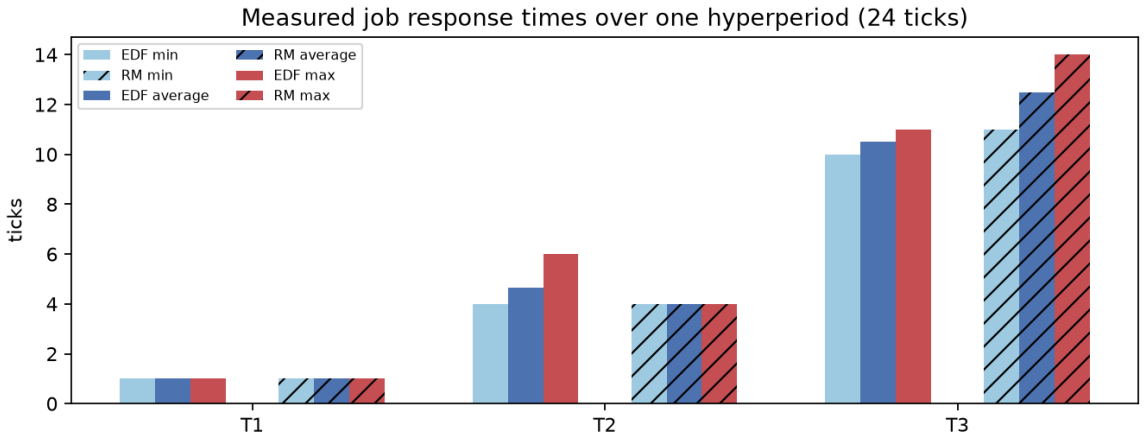


Figure 18. Minimum, average and maximum response per task under both algorithms

Utilisation, idle time and processor occupancy

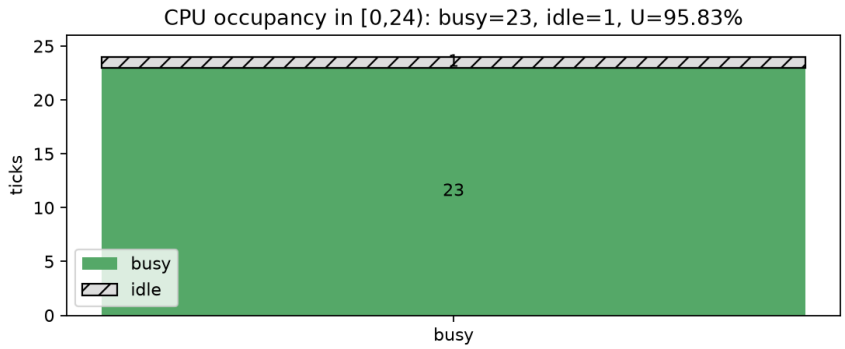


Figure 19. Processor occupancy over one hyperperiod: 23 working units, 1 idle

Hand computation of utilisation

```
                busy = 23   idle = 1   H = 24
CPU utilization = busy / H = 23 / 24 = 0.9583 = 95.83%
   offered load    = U = 23/24              = 95.83%
```

Computing utilisation from the execution trace

Why one unit of idle time exists

- The total load in the hyperperiod is 23 units and the hyperperiod is 24 units long.
- So there is exactly one time unit with nothing to do.
- In both algorithms that unit lands at the end of the hyperperiod, at tick 23.
- Idle time is not a defect; it simply means the load is below 100%.
- If the load were exactly 24 units there would be no idle time at all.

Tardiness

- Tardiness is how late a job is relative to its deadline.
- Under EDF the tardiness of every job is zero.
- Under RM the tardiness of T3_0 is 2 units.
- In a hard real-time system any positive tardiness counts as failure.

Reconciling the hand solution with the code

How the check is done

- The execution order computed by hand is stored as a string in the program output.
- The automated tests compare that string against the simulator output.
- The job count and total load are also verified automatically.
- Any disagreement between the hand solution and the code fails the tests.

```
$ python code/run_experiments.py

algorithm: EDF
  horizon=24  jobs=11  busy=23  idle=1
  utilization=95.83%  (offered 95.83%)  demand=23
  deadline misses: 0
  timeline: T1 T2 T2 T2 T1 T3 T3 T3 T1 T3 T2 T2 T1 T2 T3 T3 T1 T2 T2 T2 T1 T3 T3 -

algorithm: RM
  horizon=24  jobs=11  busy=23  idle=1
  utilization=95.83%  (offered 95.83%)  demand=23
  deadline misses: 1
    miss -> task=T3 job=0 release=0 deadline=12 finish=14
  timeline: T1 T2 T2 T2 T1 T3 T3 T3 T1 T2 T2 T2 T1 T3 T3 T3 T1 T2 T2 T2 T1 T3 T3 -
```

Actual program output, which agrees with the hand solution

Locked tests

- The first test compares the EDF execution order with the hand-worked one.
- The second checks the RM order and exactly one miss.
- The third controls the hyperperiod, utilisation and the Liu and Layland bound.
- Newer tests lock the response-time values and the per-task metrics.

Scope of the results

What has been proved

- Response-time analysis showed this set is not schedulable under RM; that is definitive.
- The condition U <= 1 showed this set is schedulable under EDF; that is also definitive.
- Both results hold for all time, not just for one hyperperiod.

What has not been proved

- Simulating one hyperperiod on its own is not a proof; it is a sample of behaviour.
- If the set were one-shot rather than periodic, the results above would not apply.
- Here the set is periodic, so hyperperiod analysis is valid.
- But if deadlines were constrained (D shorter than T), the EDF condition would no longer be sufficient.
- It is also assumed that there is no blocking, no resource sharing and no release jitter.

Precision of the claim: The claim of this report is not that EDF is always better than RM. The claim is that for this particular task set EDF is schedulable and RM is not, and that this has been confirmed both analytically and by simulation.

Scenarios that would change the result

- If C3 dropped from 4 to 2, utilisation would fall to 0.7917 and RM would probably succeed.
- If T3 rose from 12 to 24, its priority would not change but the load would drop.
- If deadlines became constrained, the processor-demand criterion would be needed.
- All of these can be tested with this same program by changing its arguments.

Conclusion

A three-task periodic set with utilisation 23/24 was designed and scheduled by hand under EDF and RM, step by step. The analytic results and the simulation agree completely.

- EDF met every deadline; the reason is that its priority is dynamic.
- RM missed one deadline: T3_0, whose deadline was 12, finished at 14.
- The cause is repeated preemption by tasks with shorter periods.
- Response-time analysis had already predicted this miss with R(T3)=14.
- Utilisation is 95.83% under both algorithms and idle time is 1 unit.

Answers to the exercise questions

Question	Answer
Which algorithm schedules the set?	EDF
How many deadline misses occur?	zero under EDF, one under RM
What is the utilisation?	23/24 ~ 95.83%
How much idle time is there?	1 unit at the end of the hyperperiod
Is this a definitive proof?	Yes, from the analytic test; not from simulation alone

Table 20. Short answers to the main questions of the exercise

Key message: The difference between EDF and RM in this example comes from a single decision: at t=12, when the deadline of T3_0 has arrived, EDF continues it while RM preempts it in favour of T1_3.

References and reproduction commands

References

- Liu, C. L., & Layland, J. W. — Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment — Journal of the ACM, 1973.
- Joseph, M., & Pandya, P. — Finding Response Times in a Real-Time System — The Computer Journal, 1986.
- Buttazzo, G. C. — Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications.
- Silberschatz, A., Galvin, P. B., & Gagne, G. — Operating System Concepts — chapter on Real-Time CPU Scheduling.

Reproduction commands

```
cd exercise-3
python code/run_experiments.py      # every number and table
python code/make_figures.py         # Gantt charts and figures
python code/edf_rm.py --tasks T1:1:4:4,T2:3:8:8,T3:4:12:12 \
                      --json out.json --table table.txt
python -m pytest tests/ -q
```

Commands for running Exercise 3

Output files

File	Description
output/experiments.json	all numbers in machine-readable form
output/experiments_summary.txt	text summary of the results
output/summary_EDF.txt	full EDF metrics
output/summary_RM.txt	full RM metrics
output/step_table_EDF.txt	step-by-step EDF table
output/step_table_RM.txt	step-by-step RM table
diagrams/gantt_edf.png	EDF Gantt chart
diagrams/gantt_rm.png	RM Gantt chart
diagrams/chart_rta.png	response-time analysis chart
diagrams/chart_response.png	response-time chart
diagrams/chart_utilization.png	processor occupancy chart
diagrams/fig_release_pattern.png	release pattern and deadlines

Table 21. Output files of Exercise 3

Blank worksheet 1: rewriting the EDF schedule by hand

The table below is for practising the EDF schedule by hand. Fill the 'Ran' column with the name of the job executed in each tick and note the remaining execution time of the ready jobs.

Tick	Ran (EDF)	Chosen absolute deadline	Notes
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			

Table 22. Blank worksheet for the EDF schedule — part one

Blank worksheet 2: EDF continued and the RM schedule

Tick	Ran (EDF)	Notes
20		
21		
22		
23		

Table 23. EDF worksheet continued

Now repeat the exercise for RM. Remember that under RM the priority is fixed and the task with the shorter period always wins.

Tick	Ran (RM)	Chosen absolute deadline	Notes
0			
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

Table 24. Blank worksheet for the RM schedule — part one

Blank worksheet 3: RM continued and the deadline checklist

Tick	Ran (RM)	Notes
16		
17		
18		
19		
20		
21		
22		
23		

Table 25. RM worksheet continued

After completing the schedule, fill in the table below to check the deadline of every job.

Job	Release	Deadline	Finish	Status
T1_0	0	4		
T1_1	4	8		
T1_2	8	12		
T1_3	12	16		
T1_4	16	20		
T1_5	20	24		
T2_0	0	8		
T2_1	8	16		
T2_2	16	24		
T3_0	0	12		
T3_1	12	24		

Table 26. Empty deadline checklist for both algorithms

How to use: The correct answers for these worksheets are given in the earlier pages and in the files under output/. For self-testing, fill the tables by hand first and then compare them with the program output.